ENACT Visualisation

Participant Handout

RAPP Lab · Robots, Art, People and Performance

You are going to build live visuals from a robot that **watches a person and decides how to move**. This stream lets you tap into three things at once:

- what the robot **sees** (the person)
- what is happening inside the robot's **"mind"** (the **ENACT** neural network)
- what the robot's **body** is doing

Your job is the *visuals*. Getting the data is already done for you. You pick a **channel**, you get clean numbers, you make something beautiful.

> **No machine-learning background needed.** Each data source below is explained at three levels: 🟢 **the simple idea**, 🔵 **a bit more technical**, and 🟣 **the math** (totally optional). Read as deep as you like.

Meet ENACT, the robot's brain

The neural network driving the robot is called **ENACT**, short for *Embodied Neural Action Chunking Transformer*. It is a **vision-action model (VAM)**: it takes in what the camera sees of a person and decides how the robot should move. In one breath: ENACT watches the **last 10 snapshots** of the person's skeleton, runs them through a **transformer encoder** (which relates those moments to one another), and a **transformer decoder** then plans the **next 10 moves all at once**, which is the "action chunking" part, by *attending back* to what it just saw.

The data sources below let you look inside ENACT at each stage: where it is **looking** (attention), its inner **"thoughts"** (hidden state), and **what about you** mattered (saliency). Figure 1 shows where each channel is tapped off the pipeline.

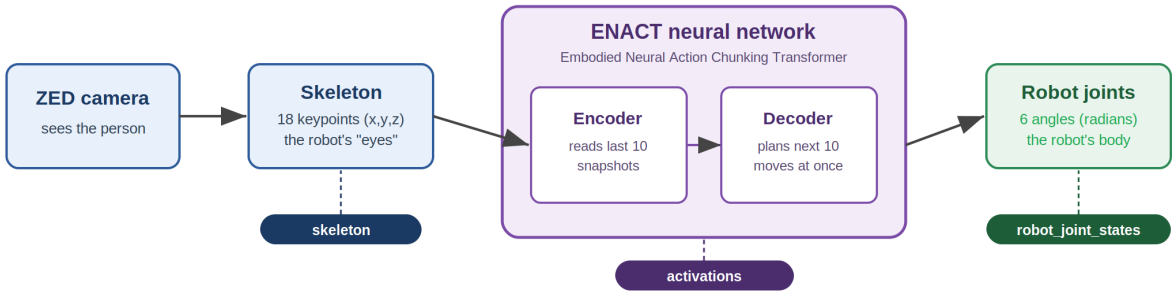

Figure 1. *The data pipeline, from camera to robot. The three labelled tags are the channels you read in Unity.*

Figure 2 unpacks the "watch 10, plan 10" idea, and shows where cross-attention links each planned move back to the input moments that mattered.

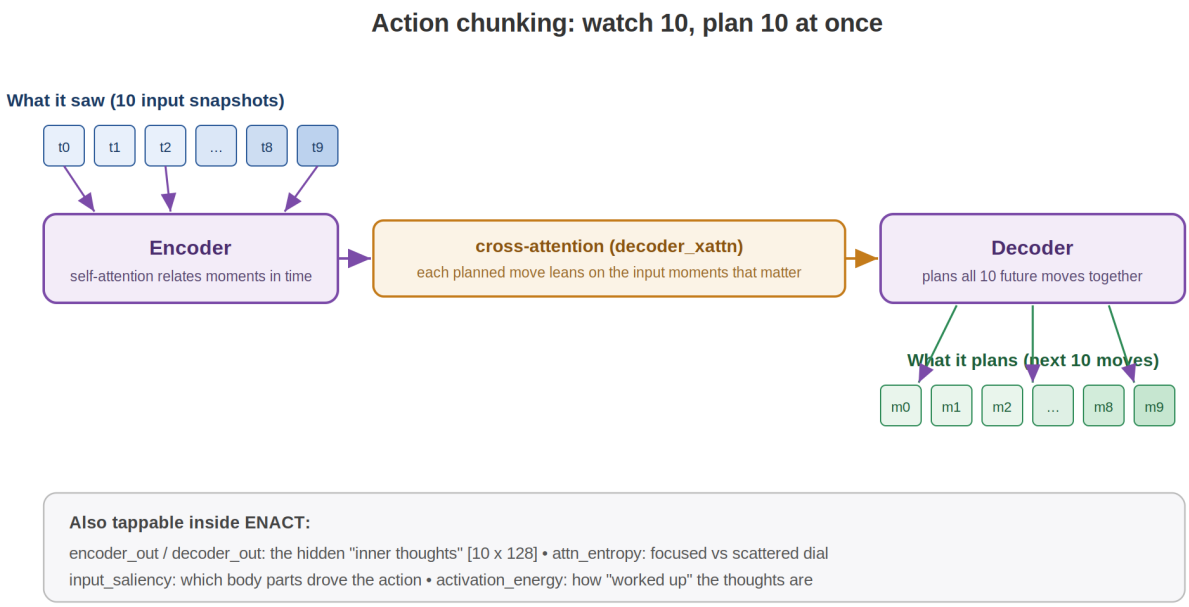

Figure 2. *Action chunking. The encoder reads ten input snapshots; the decoder plans ten future moves together; cross-attention (decoder_xattn) links the two.*

Data sources at a glance

Channel	What it is	Shape	Think of it as…
`skeleton`	the tracked person's keypoints	`[18,3]`	the robot's eyes (what it sees of you)
`activations → input_saliency`	input attribution	`[10,K]`	which parts of you drive the action
`activations → decoder_xattn`	cross-attention	`[2,4,10,10]`	what the robot focuses on to decide each move
`activations → encoder_selfattn`	self-attention	`[3,4,10,10]`	how it connects moments in time
`activations → encoder_out / decoder_out`	hidden state	`[10,128]`	the robot's inner thoughts
`robot_joint_states`	the robot's actual joint angles	`[6]`	the robot's body pose
`attn_entropy`	attention focus	`small`	a focused vs scattered dial
`activation_energy`	activation strength	`[10]`	a how-active dial

All angles are in **radians**. The `activations`, `attn_entropy` and `activation_energy` channels only stream when the instructor runs the robot's visualisation node.

1. skeleton, what the robot sees of you

🟢 **The simple idea.** This is the robot's eyes: a 3D stick-figure of the person in front of it, where the head, shoulders, elbows, hands, hips, knees and feet are in space. The camera finds these points on the body many times a second. This is the robot's whole picture of you.

🔵 **A bit more technical.** 18 body keypoints (the BODY_18 layout), each with an (x, y, z) position in metres, from the ZED camera's body tracking, an [18, 3] matrix. Drawing lines between the right pairs of points (neck to shoulder, shoulder to elbow, and so on) gives a skeleton. Figure 4 is your index reference.

🟣 **The math.** Points $p_k \in \mathbb{R}^3$. Bones are a fixed list of index pairs (a, b); draw a segment from p_a to p_b. The coordinates are in the camera's reference frame (a robotics convention: x-forward, y-left, z-up), so to place them in Unity you remap axes. The example does this for you via rosToUnity.

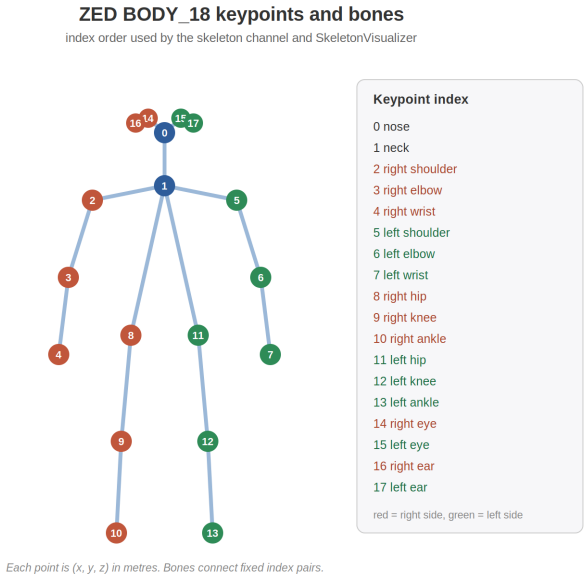

Figure 4. *The ZED BODY_18 keypoint indices and the bone pairs used by SkeletonVisualizer. Red marks the right side of the body, green the left.*

```
Shape: [18,3] — 18 keypoints x (x, y, z), in metres

k0  nose           [ 0.05,  0.10, 1.85 ]
k1  neck           [ 0.04,  0.00, 1.60 ]
k2  right shoulder [ 0.22, -0.02, 1.58 ]
k4  right wrist    [ 0.33, -0.08, 1.21 ]

A bone pair (1,2) = a line from neck k1 to shoulder k2.
```

What it represents: the robot's *perception*, its understanding of where the person is and how they are posed.

2. input_saliency, which parts of you matter (about *space*)

(channel `activations`, tensor `input_saliency`; only when saliency is enabled)

🟢 **The simple idea.** This answers: what about the person caused the robot to act? If the robot starts moving because your right hand came up, your right hand lights up. It paints the importance of each body part back onto the skeleton, the robot's way of saying “I moved because of this part of you.”

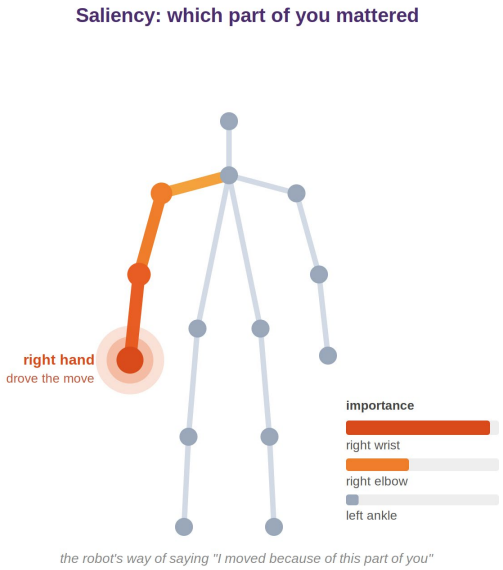

Figure 8. *Saliency painted back onto the skeleton. Here the right arm drove the action, so it lights up while the rest stays dim. The bars show the relative importance of each keypoint.*

🔵 **A bit more technical.** It is an attribution: how sensitive ENACT's chosen action is to each input keypoint. A large value means “nudge this body point a little and the action changes a lot.” It comes as a value per keypoint over the 10 input timesteps, shape [10, K].

🟣 **The math.** Gradient-based saliency: $s_k = |\partial(\text{action})/\partial(\text{input}_k)|$, summed over that keypoint's x/y/z. Bigger gradient magnitude means stronger influence on the output. (It is a first-order, local sensitivity, a good visual cue, not a precise causal proof.)

```
Shape: [10,K] — 10 timesteps x K keypoints (K=18)

right wrist    0.91   <- matters most
right elbow    0.40
right shoulder 0.18
neck           0.07
left ankle     0.03   <- barely matters right now

-> light up the right arm; leave the rest dim.
```

What it represents: the robot's *attention onto the body itself*, which parts of the human are driving its behaviour.

3. Attention, which moments matter (about *time*)

(channel `activations`*, tensors* `decoder_xattn` *and* `encoder_selfattn`*)*

🟢 **The simple idea.** When you catch a ball, you don't just see where it is now, you track how it moved over the last moment to predict where it's going. ENACT does the same with your motion: out of the last ten snapshots, it leans on the moments that tell it what you're about to do. Attention is a glowing grid where brighter means "this moment is driving my next move."

🔵 **A bit more technical.** ENACT looks at the last 10 snapshots of the person. Attention is a set of weight grids (it is a transformer):

- `Self-attention` (encoder_selfattn) lives in the encoder. The encoder's job is to read the ten input snapshots of your skeleton and relate them to one another, so both axes are the ten input timesteps. Row i, column j asks: when the model builds its understanding of moment i, how much does it lean on moment j? The bright diagonal in Figure 6 shows each moment attending mostly to itself and its neighbours, which is what "stitching your motion into one story over time" means.
- `Cross-attention` (decoder_xattn) lives in the decoder, and it is the more revealing channel. The decoder is producing the ten planned future moves, and for each one it asks: which of the ten input snapshots should I lean on? The axes are therefore asymmetric, with rows as the ten planned moves and columns as the ten input timesteps. This is why it is the clearest "why did it do that" signal: it links an output move back to the input moments that drove it. Each planned move is produced by one of ten action queries, where query 0 plans one step ahead and query 9 plans ten steps ahead, and cross-attention is how each query reaches back into what the camera saw.

Each grid is 10×10 numbers between 0 and 1; each row adds up to 1 (it is dividing 100% of its attention across the moments); there are several grids (one per head and per layer); the examples average them into one map. Figures 5 and 6 show the two attention types side by side.

🟣 **The math.** Attention is $\text{softmax}(QK^T / \sqrt{d}) \cdot V$. The part we visualise is the weight matrix $A = \text{softmax}(QK^T / \sqrt{d})$, shape [rows × 10], where each row is a probability distribution over the 10 input timesteps ($\Sigma = 1$). We expose it per layer and head: [layers, heads, rows, 10].

```
Shapes: decoder_xattn [2,4,10,10], encoder_selfattn [3,4,10,10]
        = layers x heads x rows x 10 input timesteps

One attention row (how one moment splits focus over the 10 inputs):

[0.02,0.03,0.04,0.05,0.06,0.08,0.12,0.20,0.25,0.15]  (sums to 1.00)
 t0   t1   t2   t3   t4   t5   t6   t7   t8   t9

-> most attention on t7-t9 (the most recent frames).
```

What it represents: the robot's *focus*, which moments of what it saw are driving the decision it is making now.

> **Honest caveat for the curious:** attention shows where information flows, which is a strong hint about focus but not a literal “gaze.” Great for intuition and art; do not treat it as a perfect explanation.

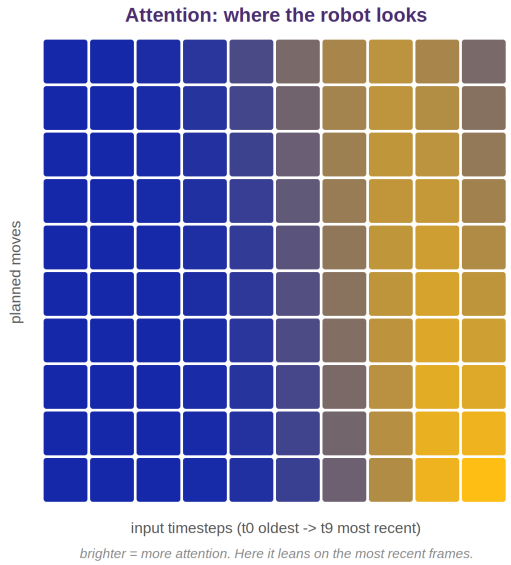

Figure 5. *Cross-attention (decoder_xattn). The axes differ: rows are the ten planned moves, columns the ten input timesteps. Each planned move leans on the input moments that mattered, here the most recent frames. This is the clearest "why did it do that" signal.*

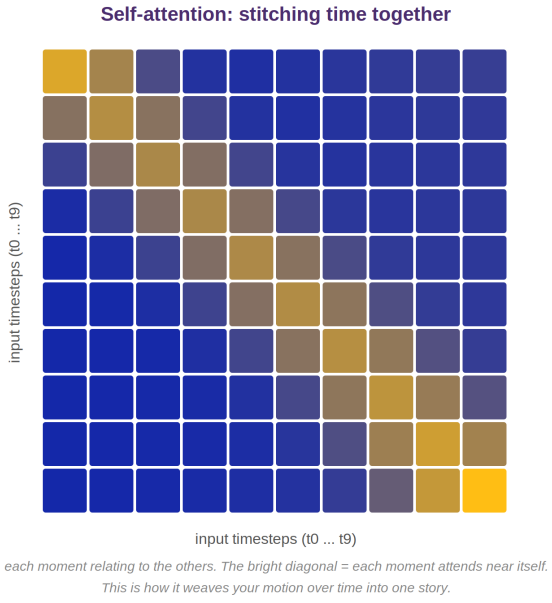

Figure 6. *Self-attention (encoder_selfattn). Both axes are the ten input timesteps, so the bright diagonal shows each moment attending near itself. This is how ENACT weaves your motion over time into one story.*

4. encoder_out / decoder_out, the robot's inner thoughts

(channel `activations`*, tensors* `encoder_out` *and* `decoder_out`*)*

🟢 **The simple idea.** After looking at you, the robot forms an inner impression, not a picture or a word, just a pattern of internal signals that shifts as the situation changes. It is like a brain scan: you cannot read a single brain cell, but you can watch the whole pattern light up, pulse and settle. encoder_out is “what I understood about the person,” and decoder_out is “the plan I am forming.”

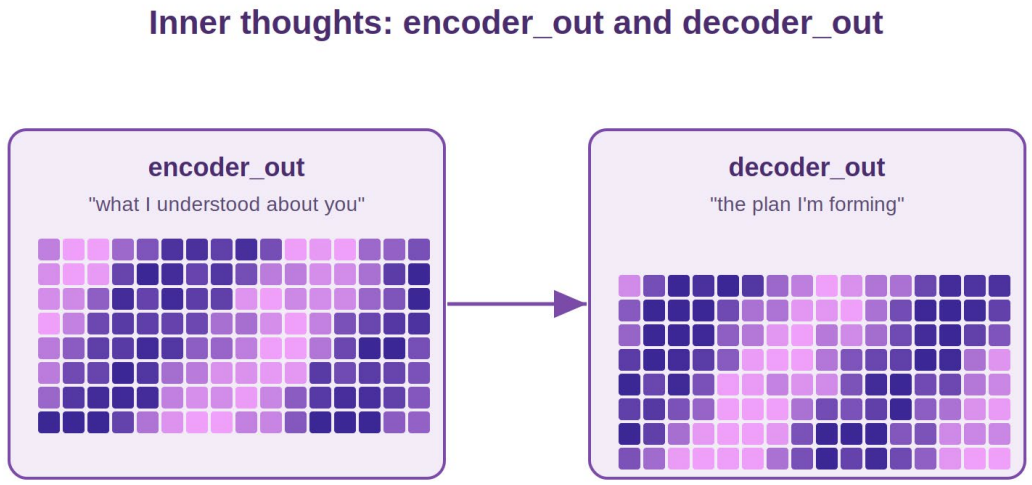

Figure 7. *Inner thoughts as two [10 x 128] grids. encoder_out is what the robot understood about you; decoder_out is the plan it is forming. Read the whole pattern, not any single cell.*

🔵 **A bit more technical.** These are ENACT's hidden activations: for each of 10 timesteps, a vector of 128 numbers, so a [10, 128] grid. encoder_out is ENACT's compressed understanding of the input; decoder_out is the representation of its planned actions, just before it turns them into joint commands.

🟣 **The math.** Vectors $h_t \in \mathbb{R}^{128}$. Individual numbers are not meaningful on their own, but the pattern is: useful views are the per-timestep magnitude $\|h_t\|$ (see activation_energy), or squashing the 128 dimensions down to 2 or 3 with PCA or t-SNE to plot the “mental state” as a moving point.

```
Shape: [10,128] — 10 timesteps x 128 features (each of enc/dec out)

h0 = [ 0.13, -0.42, 0.05, 0.88, -0.11, 0.30, -0.67, 0.24, ... ]
h1 = [ 0.10, -0.39, 0.02, 0.91, -0.08, 0.35, -0.70, 0.20, ... ]

Do not read one number; watch the whole row of 128 shift as the
person moves.
```

What it represents: the robot's *internal state*, its evolving “thoughts” about the situation.

5. robot_joint_states, the robot's body

🟢 **The simple idea.** A robot arm is made of segments connected by joints, like your shoulder, elbow and wrist. This tells you the angle of each of the robot's 6 joints right now, exactly how the arm is posed at this instant. It is the real arm's actual position, so anything you build with it stays perfectly in sync with the physical robot.

🔵 **A bit more technical.** Six numbers, one per joint, in radians, in joint order (base to tip). These are the robot's measured encoder values (feedback from the hardware), not a command, so it is ground truth for where the arm is.

🟣 **The math.** Each joint angle is $\theta_i \in \mathbb{R}$ (radians). Stacking them gives the arm's configuration $q = [\theta_1 \ldots \theta_6]$. Forward kinematics turns q into the 3D position of the hand by chaining each joint's rotation. Handy conversion: degrees = radians × 180/π.

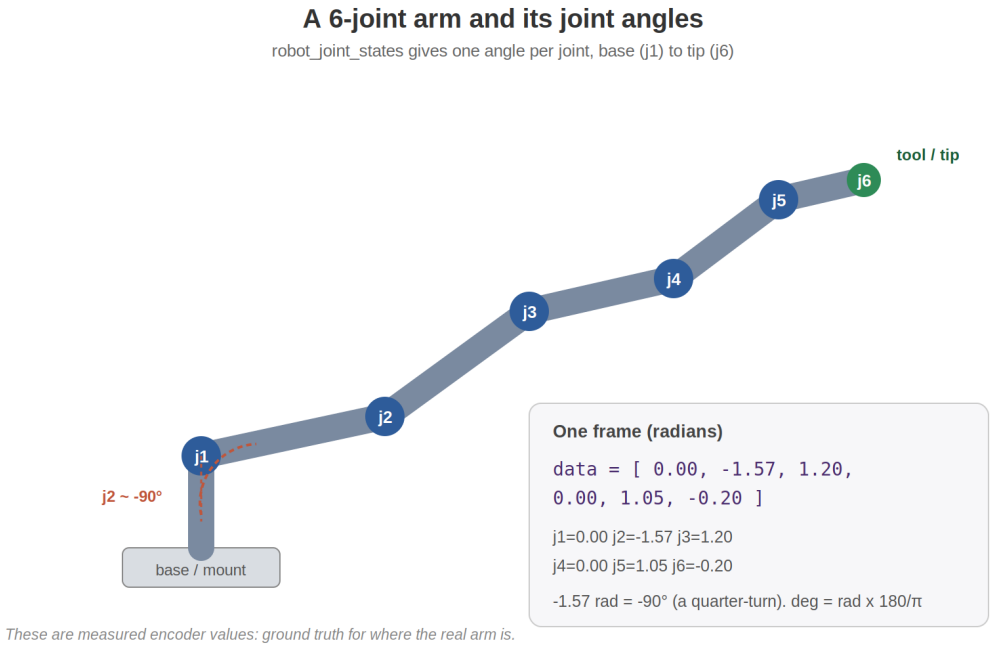

Figure 3. *The six joint angles posed from the example frame below. j2 is bent roughly a quarter-turn. The values are measured encoder readings, so they track the physical arm exactly.*

```
Shape: [6] — one angle per joint

data = [ 0.00, -1.57,  1.20,  0.00,  1.05, -0.20 ]   (radians)
         j1     j2     j3     j4     j5     j6

j2 = -1.57 rad  ~  -90 deg  (that joint is bent a quarter-turn)
```

What it represents: the robot's *body*, its physical pose.

6. attn_entropy and activation_energy, the summary dials

These are small, friendly numbers computed from the big tensors, perfect for simple meters, bars or audio. Figure 9 shows both alongside an attention map.

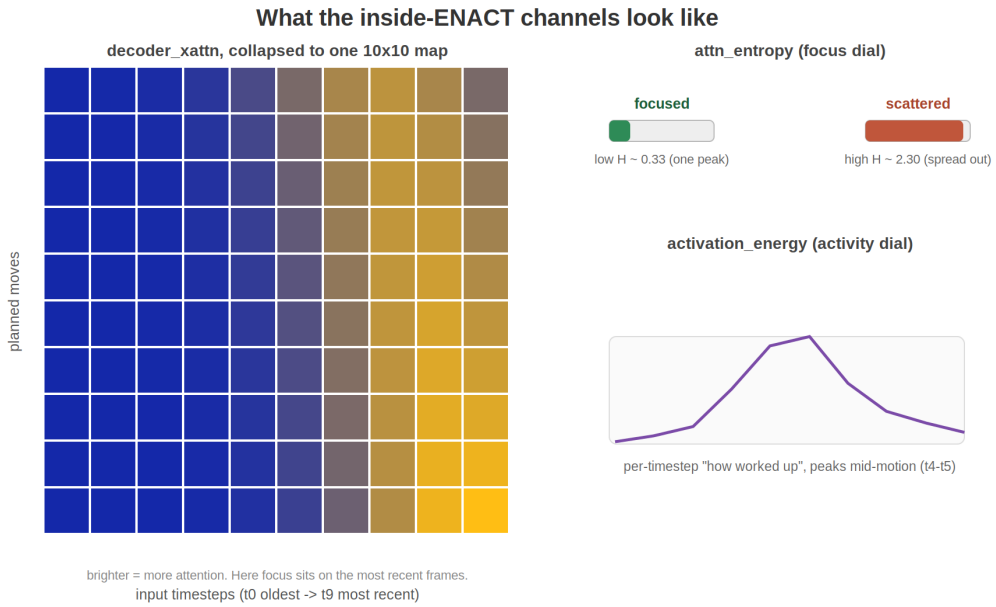

Figure 9. *Left: a decoder_xattn map collapsed to one 10x10 heatmap. Right: the two summary dials, attention focus (entropy) and activity (energy).*

attn_entropy, the focus dial

- 🟢 Is the robot laser-focused on one moment, or scanning everywhere? Low means focused; high means spread out.
- 🔵 The spread of the attention distribution, one number per layer.
- 🟣 Entropy H = −Σ p·log p over the attention weights (low H means peaky, high H means uniform). Shape: one value per layer (decoder_xattn_entropy [2], encoder_selfattn_entropy [3]).
-

```
focused   p = [0,0,0,0,0,0,0,0,0.9,0.1]  -> H ~ 0.33  (small)

scattered p = [0.1,0.1,...,0.1] (x10)     -> H ~ 2.30  (= ln 10, max)
```

activation_energy, the activity dial

- 🟢 How “worked up” the robot's thoughts are at each moment, a heartbeat for its mental state.
- 🔵 The strength (magnitude) of the hidden-state vector at each of the 10 timesteps.
- 🟣 The L2 norm $\|h_t\|_2$ per timestep. Shape: [10], one value per timestep (encoder_out_norm, decoder_out_norm).
-

```
per-timestep energy (shape [10]):

[3.1, 3.3, 3.6, 4.8, 6.2, 6.5, 5.0, 4.1, 3.7, 3.4]
 t0                  ^ peak: most worked up mid-motion        t9
```

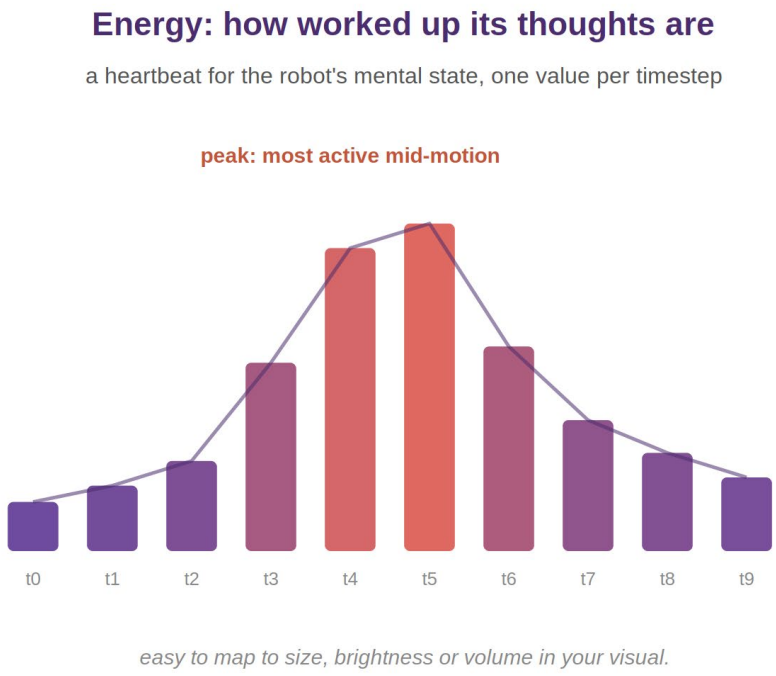

Figure 10. *Activation energy as a heartbeat: one value per timestep, peaking mid-motion when the robot's hidden state is most active. Easy to map to size, brightness or volume.*

What they represent: at-a-glance dials for *focus* and *mental activity*, easy to map to size, colour or sound.

Quickstart, get something on screen

You need: Unity (any recent version, 3D project) and the address of the stream

(`ws://<host-ip>:8765`) *or* a recording file.

The instructor gives you the address; `localhost` works if you run a recording yourself (see the last step).

1. New project and the scripts

- Download this github repo: https://github.com/maleenj/RAPP-Realtime-Viz
- Unity Hub → New Project → 3D.
- Window → Package Manager → + → Add package from git URL → `https://github.com/endel/NativeWebSocket.git#upm` → Add.

  (This is the only package you need. No JSON package, it is bundled.)
- Drag the whole `unity/` folder from the github download into your project's `Assets/`.

2. See the data immediately (the inspector)

- Create an empty GameObject, name it ENACT. Add Component → EnactClient.
- On EnactClient set Url = `ws://<host-ip>:8765`.
- Add Component → EnactInspector. Press Play.

You should see an on-screen panel listing every channel, joints as bars, attention and activation channels as little colour heatmaps. If it is there, you are connected and ready to build. If not, see Troubleshooting at the bottom.

3. Run pre-recorded data (no robot needed)

Two options:

1. Server replay (matches “live” exactly): on your machine run

   ```
   python3 player.py recordings/r1g1.jsonl --loop
   # needs: pip install websockets
   ```

   then set `EnactClient.Url = ws://localhost:8765`.

2. No Python: copy a recording into `Assets/recordings/` and rename it to end in `.txt` (e.g. `r1g1.jsonl.txt`). On EnactClient set Source = FilePlayback, drag the file into Recording, tick Loop. Press Play.

Either way the rest of your scene is identical, only the data source changes.

The template, your starting point for any visual

Before the template, it helps to see how the pieces connect. You add **one** EnactClient (the shared connection). Each visual you build gets its own EnactData component set to a single channel, then reads the numbers and draws. Figure 11 shows the pattern.

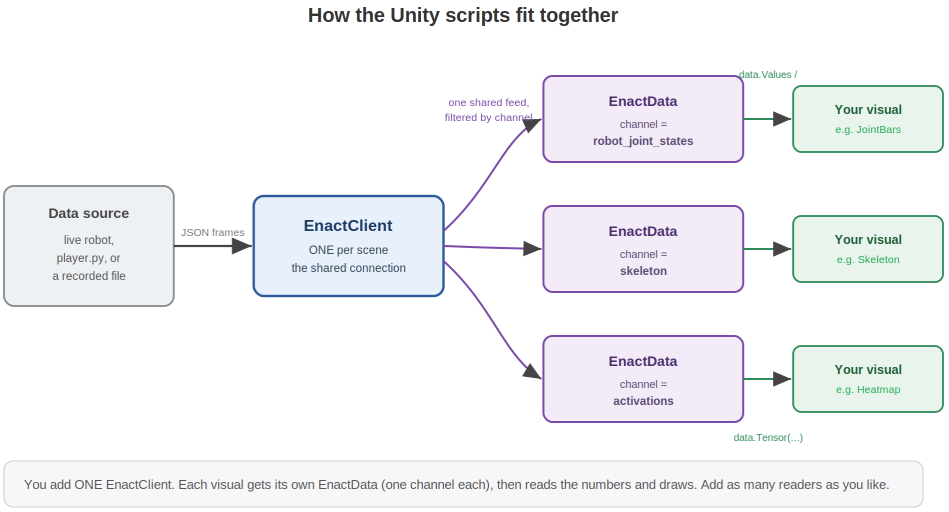

Figure 11. *The Unity script architecture. One EnactClient feeds many EnactData readers, each filtered to one channel, each driving your own visual script.*

Open `EnactVisualizerTemplate.cs`. This is the file you copy to start a new visualisation. It already does all the plumbing; you only write the visual.

How to use it

1. Duplicate `EnactVisualizerTemplate.cs`, rename the file and the class (e.g. `MyCoolViz`).
2. Put it on a GameObject. An EnactData component is added automatically.
3. On that EnactData, set Channel to the data you want (see the table near the top, e.g. `skeleton` or `activations`).
4. Fill in the marked block with your visual.

Getting the data (three easy ways):

```
EnactData data = GetComponent<EnactData>();

if (data.HasData) {
    float[] v = data.Values;                // a flat channel (joints)
    float   m = data.Get(2, 3);             // a matrix element [row, col]
    EnactTensor attn = data.Tensor("decoder_xattn");  // an activation tensor
    float[] map = attn.MeanPlane();         // collapse to one 2-D heatmap
}
```

or react only when fresh data arrives, instead of every frame:

```
data.OnData += frame => { /* runs once per new frame (~15x/sec) */ };
```

That is the whole API: `data.HasData`, `data.Values`, `data.Get(...)`, `data.Tensor(...)`, `data.OnData`. Everything else is your creativity.

The examples, read these then remix them

Each is a small, commented script. Attach the listed components and press Play.

`EnactInspector` — a zero-setup overlay of every channel (numbers, bars, heatmaps). Your debugging and exploration tool.
Attach: one GameObject with EnactClient + EnactInspector.

`JointVisualizer` — rotates 6 cubes by the joint angles. The simplest “it's alive” demo.
Attach: EnactData (channel robot_joint_states) + JointVisualizer.

`ExampleJointBars` — a row of bars that rise and fall with each joint angle.
Attach: EnactData (channel robot_joint_states) + ExampleJointBars.

`SkeletonVisualizer` — draws the tracked human as joints and bones. If it looks rotated or mirrored, toggle rosToUnity. Needs the skeleton stream running.
Attach: EnactData (channel skeleton) + SkeletonVisualizer.

`ExampleAttentionHeatmap` — a grid of tiles that glow with the robot's attention (decoder_xattn, averaged to one 10x10 map). Needs the activation stream running.
Attach: EnactData (channel activations) + ExampleAttentionHeatmap.

`FlyCamera` — put it on your Main Camera. W A S D to move, arrow up/down for height, hold right-mouse to look, Shift to go faster. (Uses Unity's legacy Input; if you get an input error, set Project Settings → Player → Active Input Handling to “Both.”)
Attach: your Main Camera.

`ConnectionStatusUI` — shows connected / rate / latest values in a corner.
Attach: anywhere.

Troubleshooting

- **Inspector says “no EnactClient found”** → add an EnactClient component to a GameObject.
- **Status never turns green / can't connect** → wrong IP, the WiFi blocks laptop-to-laptop traffic, or a firewall is blocking port 8765. Test first with the browser page (web/index.html); if that fails too, it is the network, not Unity.
- **Connected, but my channel is empty** → check the channel name matches the table exactly. skeleton, activations and so on only stream when the instructor runs the matching node on the robot; robot_joint_states is always there.
- **Skeleton looks sideways or mirrored** → toggle rosToUnity on SkeletonVisualizer, or adjust scale/offset.

Have fun. The data is yours; make something that makes people feel what the robot is thinking.